API Testing

Session-1: Introduction

Mannual – Postman

Automation – RestAssured

Client: A computer hardware device/software that accesses a service made available by server. Server is often (not always) located in another physical computer

Server: A physical computer dedicated to run services to serve needs of other computers

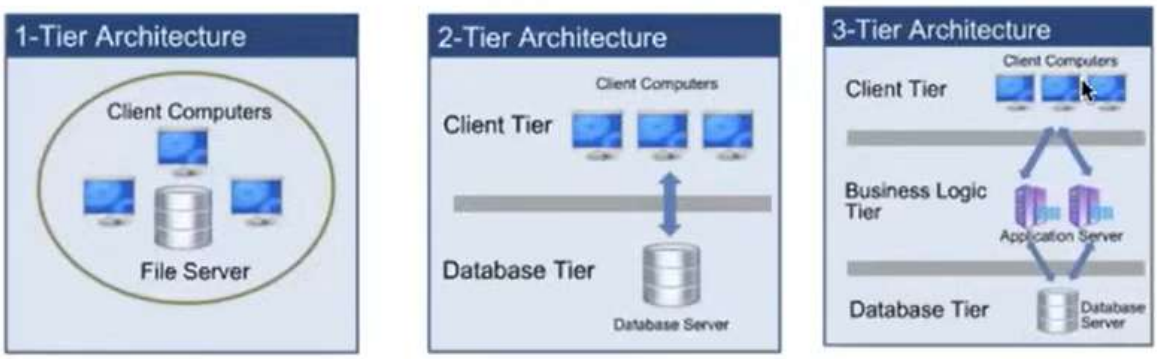
Ex: We are able to access google pages through server

Client: Google/ Mobile application requests

Server: From where software or web pages are exist

Internet: Connection between Server and Client

Client/Server Architectures



1Tier: Client and server runs in same machine

Ex: Saving data in same machine (no need internet, old tech)

2Tier: Clients and Server in different locations

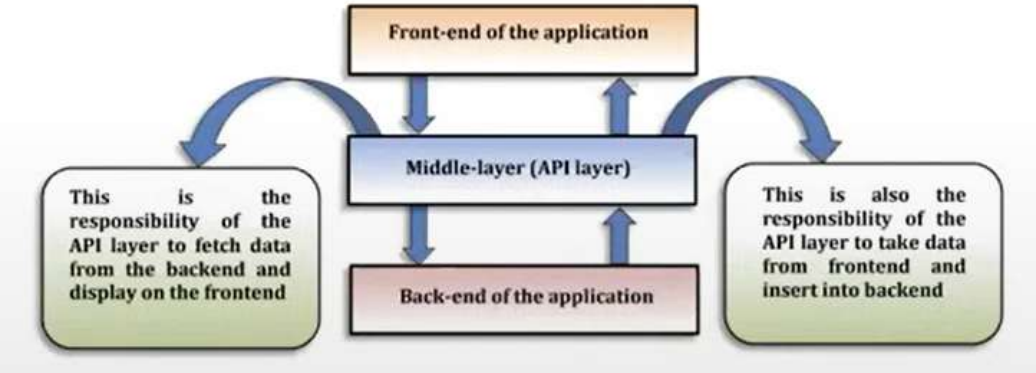
Ex: Browsing Applications

3Tier: Client Tier, Business logic Tier, Database Tier

Request: Client -> Business logic -> Data Base

Response: DataBase -> Business Logic -> Client

API- Application Program Interface – A way of communicating btwn 2 Apps



Types of API's

- Simple Object Access Protocol(SOAP)– OLD (only support xml)
- Representational State Transfer(REST)– Latest (xml, json, html, txt)

API vs WebService

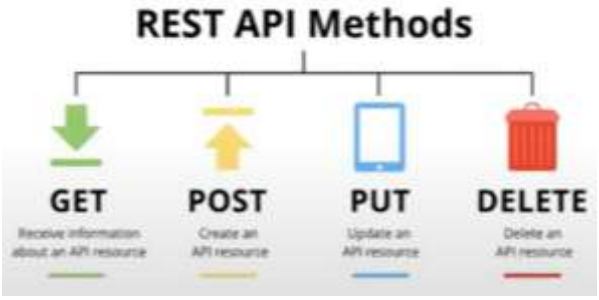
WebService - If we keep API in WEB

All Webservices are API's but All Api's are not Webservices

During dev or testing they called as API which doesn't need network

Once they available for users in web env they called as Webservices

REST API methods



GET: get response from server

POST: sending data to server to store

PUT: To update existing data from DB

DELETE: to delete data from DB

PATCH: Updating Partial details of resource

Terminologies

HTTP vs HTTPS (Encryption and verification is included)

Hyper Text Transfer Protocol (SECURE)

URI – Uniform Resource Identifier

URL – Uniform Resource Locator

URN – Uniform Resource Name

Feature and Resource

Feature – Term used in Mannual testing to test some functionality

Resource - Term used in API automation testing referring somefunctionality

Payload- Data sending along with request & data coming along with response

- Request Payload
- Response Payload

Session-2: Env Setup

https://reqres.in/api/users?page=2

- Host domain - https://reqres.in
- Path parameters - /api/users
- Query parameters – page=2

1XX Informational	
100	Continue
101	Switching Protocols
102	Processing

2XX Success	
200	OK
201	Created
202	Accepted
203	Non-authoritative Information
204	No Content
205	Reset Content
206	Partial Content
207	Multi-Status
208	Already Reported
226	IM Used

3XX Redirectional	
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
305	Use Proxy
307	Temporary Redirect
308	Permanent Redirect

4XX Client Error	
400	Bad Request
401	Unauthorized
402	Payment Required
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required
408	Request Timeout

4XX Client Error Continued	
409	Conflict
410	Gone
411	Length Required
412	Precondition Failed
413	Payload Too Large
414	Request-URI Too Long
415	Unsupported Media Type
416	Requested Range Not Satisfiable
417	Expectation Failed
418	I'm a teapot
421	Misdirected Request
422	Unprocessable Entity
423	Locked
424	Failed Dependency
426	Upgrade Required
428	Precondition Required
429	Too Many Requests
431	Request Header Fields Too Large
444	Connection Closed Without Response
451	Unavailable For Legal Reasons
499	Client Closed Request

5XX Server Error	
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported
506	Variant Also Negotiates
507	Insufficient Storage
508	Loop Detected
510	Not Extended
511	Network Authentication Required
599	Network Connect Timeout Error

HTTP STATUS CODES

When a browser requests a service from a web server, an error may occur.
This is a list of HTTP status messages that might be returned.

Session-3: To Create Own API's

Install NodeJs (includes npm (Node Package Manager) package)

npm is world's largest software registery

Json-server: **npm install -g json-server**

JSON - Java Script Object Notation – Data format ,light weight & encrypted

Create a students.json file

```
{
 "students":[
        {
        "id":1,
        "name":"John",
        "courses":["Java","Selenium"]
        },
        {
        "id":2,
        "name":"Kim",
        "courses":["Python","Maven"]
        },
        {
        "id":3,
        "name":"Tom",
        "courses":["C","Gradle"]
        }]
}
```

Run **json-server students.json** in cmd

```
C:\Library\API>json-server students.json
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching students.json...

( ·.· ▢ ·.· )

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/students
```

JSON:

- Java Script Object Notation
- Syntax for storing and exchanging data
- xpath is derived from xml & json extended from javaScript
- internet media type application/json
- Data Types {Number, String, Boolean, Null, Object, Array}

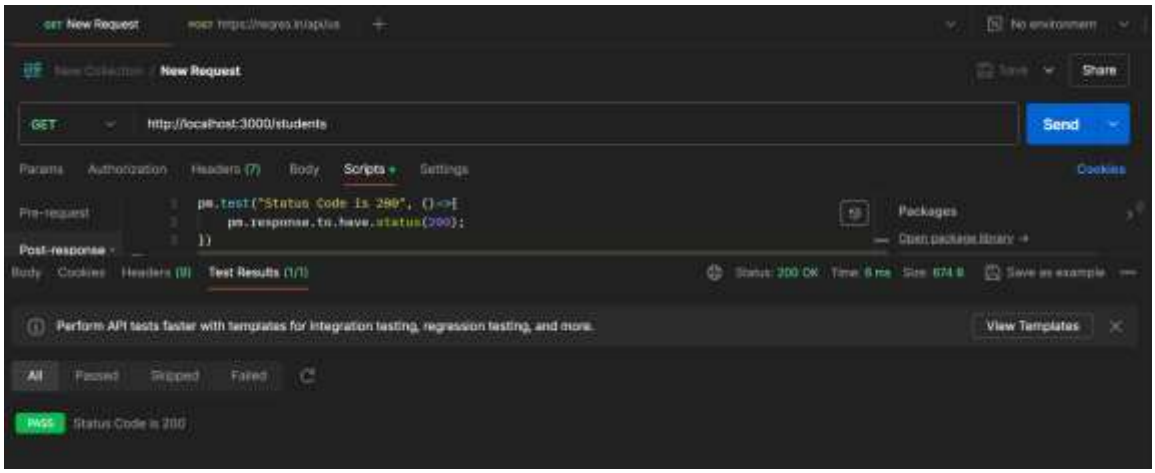
JSON vs XML

JSON	XML
JSON is simple to read and write.	XML is less simple as compared to JSON.
It also supports **array**.	It doesn't support array.
JSON files are more **human-readable** than XML.	XML files are **less human readable**.
It supports only **text** and **number** data type	It supports many data types such as **text, number, images, charts, graphs**, etc.

Sesion-4: Response Validations

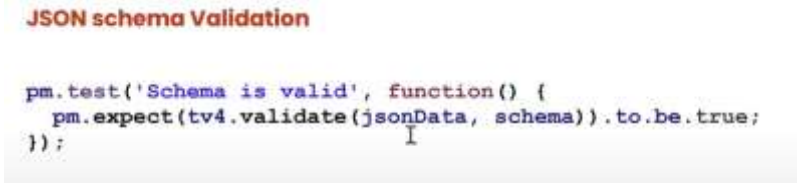
- Status Code
- Headers
- Cookies
- Response Time
- Response Body

Chai Assertion Library – A JS framework library

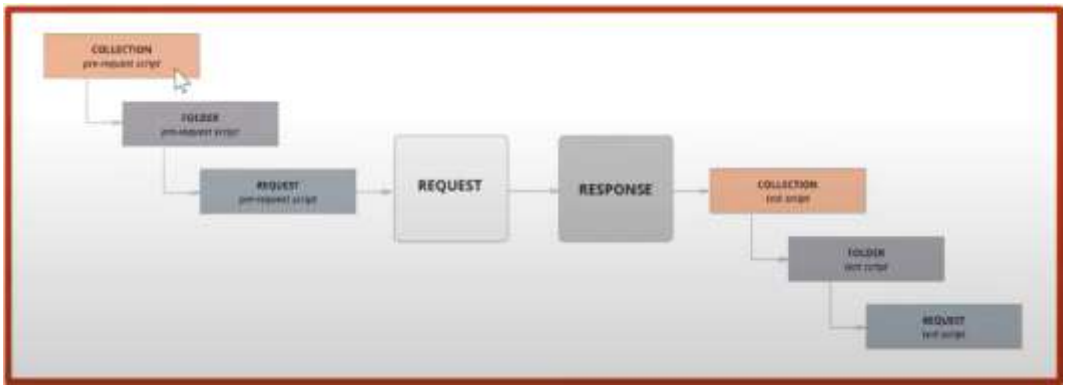



```
//Testing headers
pm.test("Content-Type header is present", () =>{
    pm.response.to.have.header("Content-Type");
});

//Testing cookies
pm.test("Cookie language is present", () =>{
    pm.expect(pm.cookies.has("language")).to.be.true;
});

//Asserting Response time
pm.test("Response Time less than 10ms", () =>{
    pm.expect(pm.response.responseTime).to.be.below(10);
});

//Testing Response Body
var jsonData = pm.response.json();
pm.test("Test data type of response", () =>{
    pm.expect(jsonData).to.be.an(Object);
    pm.expect(jsonData.courses).to.eql("Java");
});
```

Json Schema – Meta data about data



Session-5: Scripts & Types of Variables



Variables: Based on scope of access

- Global – Variable can access throughout the workspace
- Collection – accessible within the collection
- Environment – access to all collections within specified environment
- Local – Declared in pre-requests script (specific to request)
- Data – external files csv/text

Syntax – {{var_name}}

```
//set-to create
pm.globals.set("id_global","1");
pm.environment.set("id_collection","2");
pm.collectionVariables.set("id_env","3");
pm.variables.set("id_local","4");
//get-to retrieve
pm.globals.get("id_global");
//unset - to delete
pm.variables.unset("id_local");
```

Session-6: API Chaining

Response of one API will become request for another API

GoRest: https://gorest.co.in/ //realtime online server

Passing Id from prev API's response to current API's GET request

Session-7: Parameterisation

Data driven Testing - Passing json file/csv file at the time of run iterations.

Session-8: Authentication & File Upload API

Basic Auth – Username & password are not encrypted

Digest Auth – Encrypted username and password

OATH 2.O -

Step-1: send client ID to get Code

Step-2: send client ID , client secret & code to get Token

Step-3: Use Token for API calls

Session-9: Swagger

Swagger – Interactive Documentation

Just for exploring API's not for testing

we can't add validations

we can't save or generate reports from results

cURL – Client URL

Command line tool which developers use to transfer data to & from server.

Session-10: Run Collections from CLI

npm install -g newman //installing newman from npm

npm install -g newman-reporter-html // installing newman-reporter

newman run <exported_collection_name.json> // to run in CLI

newman run <exported_collection_name.json> -r html // for html report

Can also run from Jenkins

Session-11: Rest Assured

It is an API/Library through which we can automate REST API's

Dependencies:

- rest-assured
- json-path
- json
- gson
- testng
- scribejava-apis //sometimes useful to pass random data
- json-schema-validator
- xml-schema-validator

RestAssured by default supports BDD Gherkin language.

.given()//content type, set cookies, add auth, add param, set headers…

.when()//get, put, post, delete

.then()//validations

.and()//part of then used for multiple validations

Below 3 static packages need to add in all java files, then only key words will identified

```
import static io.restassured.RestAssured.*;
import static io.restassured.matcher.RestAssuredMatchers.*;
import static org.hamcrest.Matchers.*;
```

```
public class RestAssured{
    void getUsers(){
        given()
        .when()
            .get("https://reqres.in/api/users?page=2")
        .then()
            .statusCode(200)
            .body("page",equalTo(2))
            .log().all();
    }
}
```

For PostRequest:

```
    void createUser(){
        HashMap data = new HashMap<>();
        data.put("name", "John");
        given()
            .contentType("application/json")
            .body(data);
        .when()
            .post("https://reqres.in/api/users")
        .then()
            .statusCode(201)
            .log().all();
    }
```

Creating Post Request in Multiple Ways with diff types of Bodies

- HashMap
- org.json
- using POJO (Plain Old Java Object)
- using external json file

1 - Using HashMap:

```
    void createUserUsingHashMap(){
        HashMap data = new HashMap<>();
        data.put("name", "John");
        String coursesArr[] = {"C","Java"};
        data.put("courses",coursesArr);
        given()
            .contentType("application.json")
            .body(data);
        .when()
            .post("https://reqres.in/api/users")
        .then()
            .statusCode(201)
            .log().all();
    }
```

2 - Using Org.json:

Add org.json dependencies in either maven or gradle

Like Hashmap we can't pass json obj data to body..
we need to convert to string format.

```
    void createUserUsingOrgJson(){
        JSONObject data = new JSONObject();
        data.put("name", "John");
        String coursesArr[] = {"C","Java"};
        data.put("courses",coursesArr);
        given()
            .contentType("application.json")
            .body(data.toString);
        .when()
            .post("https://reqres.in/api/users")
        .then()
            .statusCode(201)
            .log().all();
    }
```

3 - Using POJO:

using encapsulation - wrapping of variables and methods into single class suing getters and setters

Create a POJO class to generate and get data

```
package restAssured;

public class POJO {
    String name;
    String courses[];
```

```
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String[] getCourses() {
        return courses;
    }
    public void setCourses(String[] courses) {
        this.courses = courses;
    }
}
```

Now in method:

```
void createUserUsingPOJO(){
        POJO data = new POJO();
        data.setName("John");
        String coursesArr[] = {"C","Java"};
        data.setCourses(coursesArr);
        given()
            .contentType("application.json")
            .body(data);//toString only used for JSON Library
        .when()
            .post("https://reqres.in/api/users")
        .then()
            .statusCode(201)
            .log().all();
    }
```

4 - Using External JSON File:

```
    void createUserUsingExternalJSON(){
       File f = new File("\\.body.json");//capturing the file with location
       FileReader fr = new FileReader(f);//reading file
       JSONTokener jt = new JSONTokener(fr);//passing into JSON tokener
       JSONObject data = new JSONObject(jt);//passing JSON tokener to get OBJ
        given()
            .contentType("application.json")
            .body(data.toString);//needs ToString bcoz JSON data
        .when()
            .post("https://reqres.in/api/users")
        .then()
            .statusCode(201)
            .log().all();
    }
```

Query and Path Params

After Question mark - Query Parameters – like key and value pairs
Before Question mark - Path parameters – like variables

```
    void testQueryAndPathParams(){
       //https://reqres.in/api/users?page=2&id=5
        given()
            .pathParam("mypath","users")
            .queryParam("page",2)
            .queryParam("id",5)
        .when()
            .get("https://reqres.in/api/{mypath}")//no need to pass query
            .post("https://reqres.in/api/users")
        .then()
            .statusCode(201)
            .log().all();
    }
```

Query parameters can go along with path parameters automatically

Cookies and Headers:

```
    void testCookiesHeaders(){
       Response res = given()
                      .when
                         .get("https://www.google.com");
        Map<String,String> cookies =  res.getCookies();//to print all cookies
        for(String k : cookies.keySet()){//get all keys set
            String cookie_val = res.getCookie(k);//gets value of k keys
            System.out.println(k+" : "+cookie_val);
        }
        Headers myHeaders = res.getHeaders();//creating myHeaders obj
        for(Header hd:myHeaders){
            System.out.println(hd.getName()+" : "+hd.getValue());
        }
    }
```

Log:

```
        .then
            .log.all();//all response
            .log.body();//only body
            .log.cookies();//only cookies
            .log.headers();//only headers
```

Parsing Response Body:

```
void responseValidation(){
    Response res =
    given()
        .contentType(ContentType.JSON)
    .when
```

```
        .get("http://localhost:3000/store");
    //converting res to JsonObj type
    JSONObject jObj = new JSONObject(res.toString());
    Double price =0;
    for(int i=0; i<jObj.getJSONArray("book").length();i++){
        String bookTitle =
jObj.getJSONArray("book").getJSONObject(i).get("title").toString();
        String bookPrice =
jObj.getJSONArray("book").getJSONObject(i).get("price").toString();
//using jObj getting bookArray title with index i and converting toString
        System.out.println(price+Double.parseDouble(bookPrice));
//converting string to Double
        System.out.println(bookTitle);
    }   // Assert.assertEquals(res.getStatusCode(),200);
        // Assert.assertEquals(res.header("Content-Type"),"application/json;
charset=uft-8");
        // String bookname = res.jsonPath().get("book[3].title").toString();
        // //getting book3 title and storing it as a string in var
        // Assert.assertEquals(bookname,"The LORD");
    }
```

For Json we use jsonPath() and for xml we use xmlPath()

For Json – JSONObject class
For XML – XMLPath class to create obj

toString() – converts a part of data into string format

asString() – converts entire response into string format

Uploading/Downloading files

Uses inbuilt multiPart method to upload files

```
    void FileUpload(){
        File myFile1 = new File(../folder/test1.txt);
        File myFile2 = new File(../folder/test2.txt);
        given()
            .multiPart("files",myFile1)//multiPart is inbuild method which
represents form data in postman
            .multiPart("files",myFile2)
            .contentType("multipart/form-data")//content type is mandatory
        .when()
            .post("http://localhost:8080/uploadFiles")
        .then()
            .statusCode(200);
    }
```

Schema validation:

Json Response (.json) – Json schema (.json)

XML Response (.xml) – xml schema (.xsd) – XML Schema Document

For JSON we have JSONSchemaValidator class but for XML we use RESTAssuredMatchers library importing for validating XML schema indirectly

Serialization – POJO → JSON

De-serialization – JSON → POJO (Plain Old Java Object)

Using Jackson Api library and ObjectMapper class for serialization and De-serialization

POJO class nothing but class contains variabels with getters and setters

```
    void convertPOJOtoJSON(){
        Student studentPOJO = new Student();
        studentPOJO.setName("John");
        ObjectMapper objMapper = new ObjectMapper();
        //serialization
        String jsondata =
objMapper.writerWithDefaultPrettyPrinter().writeValueAsString(studentPOJO);
    }
```

```
    void convertJSONtoPOJO(){
        String jsondata = "{\r\n"
            + " \"name\" : \"John\""
            +"}";
        ObjectMapper objMapper = new ObjectMapper();
        //De-serialization
        Student studentPOJO = objMapper.readValue(jsondata,Student.class);
    }
```

Authentication: User is valid or not

- Bacic
- Digest
- Preemptive – combination of basic and digest

Authorization: User have access or not

- Bearer Token
- oauth 1.o and 2.o
- API Key

API Chaining:

Response of one API will become request of another API